
Module 14 — Testing Spring Boot Applications

Lesson Notes

Spring Boot Edu



2026-09-24

Contents

1. Learning Goals	2
2. Concepts	2
2.1 The Test Pyramid	2
2.2 What a Slice Loads	3
2.3 Test Doubles	3
3. Step-by-Step Examples	4
3.1 Unit Tests: Plain Java	4
3.2 Unit Tests with Mockito	6
3.3 Web Slice: @WebMvcTest and MockMvcTester	7
3.4 Data Slice: @DataJpaTest against PostgreSQL	7
3.5 JSON Slice: @JsonTest	8
3.6 Integration Tests: @SpringBootTest	9
3.7 Testcontainers	11
3.8 Architecture Tests with ArchUnit	11
3.9 Test Data with Builders	12
4. Common Mistakes and Best Practices	13
5. Summary	14
6. Further Reading	14

1. Learning Goals

By the end of this module you will be able to:

- Explain the test pyramid and choose the right test level for a piece of code
- Write fast unit tests with JUnit 6, AssertJ and Mockito
- Use slice tests (`@WebMvcTest`, `@DataJpaTest`, `@JsonTest`) to test one layer with its real Spring configuration
- Replace or spy on beans with `@MockitoBean` and `@MockitoSpyBean`
- Write integration tests with `@SpringBootTest`, `RestTestClient` and `Testcontainers`
- Keep test data readable with builders
- Protect the architecture with ArchUnit rules

Prerequisites: Module 06 (JPA) · **Estimated time:** 5 hours · **Docker required** for the database tests

i Note

In this module, the lesson **is the test suite**. The application (an order service) is only the object under test. Run everything with `./mvnw -pl modules/14-testing/lesson verify`, then read the tests next to these notes.

2. Concepts

2.1 The Test Pyramid

▲ integration	(<code>@SpringBootTest</code> + real DB)	few, slow, prove that parts fit
▲▲▲ slices	(<code>@WebMvcTest</code> , <code>@DataJpaTest</code> , ...)	some, one layer each
▲▲▲▲ unit	(JUnit, Mockito, no Spring)	many, milliseconds

The lower a test sits, the faster and more precise it is. The higher it sits, the more it proves about the real system. A healthy suite has many unit tests, some slice tests and a few integration tests. The times of this module's suite (one run on the author's machine):

Test	Level	Time
PriceCalculatorTest (8 cases)	unit	0.06 s
OrderServiceTest	unit with Mockito	1.1 s
OrderControllerTest	<code>@WebMvcTest</code> slice	0.9 s
OrderConfirmationJsonTest	<code>@JsonTest</code> slice	1.4 s

Test	Level	Time
BookRepositoryTest	@DataJpaTest + PostgreSQL	5.5 s (container start included)
OrderFlowIT	@SpringBootTest, real port and database	7.7 s

2.2 What a Slice Loads

A slice test starts a **part** of the application context: only the beans that belong to one layer.

Annotation	Loads	Does not load
@WebMvcTest(X.class)	controller X, MVC config, Jackson, validation, MockMvcTester	services, repositories, database
@DataJpaTest	entities, repositories, EntityManager, Flyway, TestEntityManager	controllers, services
@JsonTest	Jackson with the application's configuration, JacksonTester	everything else
@SpringBootTest	everything	—

Missing collaborators are provided with @MockitoBean.

2.3 Test Doubles

Double	What it is	In this module
Mock	records calls, returns what you program	@Mock, @MockitoBean PaymentClient
Spy	the real object, whose calls are also recorded	@MockitoSpyBean AuditLog
Fake	a simple working implementation	an in-memory store (modules 12, 13)

Mock only what you do not own or cannot run (a payment provider). Use the real thing when it is cheap (the PriceCalculator, a Testcontainers database).

3. Step-by-Step Examples

The object under test is an order service: it finds a book, checks the stock, calculates the price, charges the payment service and records an audit entry.

```
@Service
public class OrderService {

    private final BookRepository books;
    private final PriceCalculator prices;
    private final PaymentClient payments;
    private final AuditLog audit;

    public OrderService(BookRepository books, PriceCalculator prices, PaymentClient
        ↪ payments, AuditLog audit) {
        this.books = books;
        this.prices = prices;
        this.payments = payments;
        this.audit = audit;
    }

    @Transactional
    public OrderConfirmation placeOrder(String isbn, int quantity) {
        Book book = books.findByIsbn(isbn).orElseThrow(() -> new
            ↪ BookNotFoundException(isbn));
        book.removeFromStock(quantity); // may throw
        ↪ OutOfStockException
        BigDecimal total = prices.total(book.getPrice(), quantity);
        PaymentReceipt receipt = payments.charge(new PaymentRequest("order-" + isbn,
        ↪ total));
        audit.record("ordered " + quantity + " × " + isbn + " for " + total);
        return new OrderConfirmation(isbn, book.getTitle(), quantity, total,
            ↪ receipt.id());
    }
}
```

3.1 Unit Tests: Plain Java

The price rules live in a class without any Spring dependency:

```

public class PriceCalculator {

    public BigDecimal total(BigDecimal unitPrice, int quantity) {
        if (quantity <= 0) {
            throw new IllegalArgumentException("quantity must be positive, was " +
                ↪ quantity);
        }
        BigDecimal gross = unitPrice.multiply(BigDecimal.valueOf(quantity));
        BigDecimal discount = quantity >= 10 ? new BigDecimal("0.10")           // 10 %
            ↪ from 10 copies
            : quantity >= 5 ? new BigDecimal("0.05")                         // 5 %
            ↪ from 5 copies
            : BigDecimal.ZERO;
        return gross.subtract(gross.multiply(discount)).setScale(2,
            ↪ RoundingMode.HALF_UP);
    }
}

```

Its test needs nothing but new. A parameterized test covers every rule, including the boundaries:

```

class PriceCalculatorTest {

    private final PriceCalculator calculator = new PriceCalculator();

    @ParameterizedTest(name = "{1} × {0} = {2}")
    @CsvSource({
        "10.00, 1, 10.00",           // no discount
        "10.00, 4, 40.00",
        "10.00, 5, 47.50",           // 5 % from 5 copies
        "10.00, 10, 90.00",          // 10 % from 10 copies
        "89.90, 3, 269.70",
        " 9.99, 5, 47.45"})          // 49.95 - 2.4975 = 47.4525 → rounded half up
    void appliesQuantityDiscounts(String unitPrice, int quantity, String expected) {
        assertThat(calculator.total(new BigDecimal(unitPrice),
            ↪ quantity)).isEqualToComparingTo(expected);
    }

    @ParameterizedTest
    @ValueSource(ints = {0, -1})
    void rejectsQuantitiesBelowOne(int quantity) {
        assertThatIllegalArgumentException().isThrownBy(() ->
            ↪ calculator.total(BigDecimal.TEN, quantity));
    }
}

```

3.2 Unit Tests with Mockito

OrderService has collaborators. In a unit test they are **mocks**, still without any Spring context:

```
@ExtendWith(MockitoExtension.class)
class OrderServiceTest {

    @Mock
    BookRepository books;

    @Mock
    PaymentClient payments;

    @Mock
    AuditLog audit;

    @Captor
    ArgumentCaptor<PaymentRequest> paymentRequest;

    private OrderService service() {
        return new OrderService(books, new PriceCalculator(), payments, audit); // the
        ↪ real calculator
    }

    @Test
    void chargesTheDiscountedTotalAndLowersTheStock() {
        Book book = aBook().isbn("9780134685991").price("10.00").stock(8).build();
        given(books.findByIsbn("9780134685991")).willReturn(Optional.of(book));
        given(payments.charge(any())).willReturn(new PaymentReceipt("pay-1", "PAID"));

        OrderConfirmation confirmation = service().placeOrder("9780134685991", 5);

        then(payments).should().charge(paymentRequest.capture());
        assertThat(paymentRequest.getValue().amount()).isEqualByComparingTo("47.50");
        assertThat(confirmation.paymentId()).isEqualTo("pay-1");
        assertThat(book.getStock()).isEqualTo(3);
    }
}
```

- `given(...).willReturn(...)` programs a mock, `then(mock).should()` verifies a call (BDD style).
- An `ArgumentCaptor` catches the argument of a call, here the amount sent to the payment service.
- The `PriceCalculator` is real: it is cheap and deterministic, so there is no reason to mock it.
- Constructor injection makes all this possible: the test calls the constructor itself.

3.3 Web Slice: @WebMvcTest and MockMvcTester

```
@WebMvcTest(OrderController.class)
class OrderControllerTest {

    @Autowired
    MockMvcTester mvc;

    @MockitoBean // replaces the bean in the Spring
    ⇨ context with a mock
    OrderService orders;

    @Test
    void aValidOrderIsCreated() {
        given(orders.placeOrder("9780134685991", 2)).willReturn(new OrderConfirmation(
            "9780134685991", "Effective Java", 2, new BigDecimal("179.80"), "pay-7"));

        assertThat(post("""
            {"isbn": "9780134685991", "quantity": 2}"""))
            .hasStatus(201)
            .hasHeader("Location", "/api/orders/pay-7")
            .bodyJson().extractingPath("$.total").isEqualTo("179.80");
    }

    @Test
    void anInvalidBodyIs400AndNeverReachesTheService() {
        assertThat(post("""
            {"isbn": "", "quantity": 0}"""))
            .hasStatus(400);
        then(orders).should(never()).placeOrder(anyString(), anyInt());
    }
}
```

- Only OrderController and the MVC infrastructure start. The service is a @Mockito-Bean.
- MockMvcTester (Spring Framework 6.2+) sends requests without a server and offers AssertJ assertions: status, headers, JSON paths.
- The test checks what only the web layer can get wrong: validation (400), status codes, headers, error bodies.

3.4 Data Slice: @DataJpaTest against PostgreSQL

```
@DataJpaTest
@Import(TestcontainersConfiguration.class)
class BookRepositoryTest {
```

```

@Autowired
BookRepository books;

@Autowired
TestEntityManager entities; // writes test data directly, bypassing
↪ the repository

@Test
void findsABookByIsbn() {
    entities.persistAndFlush(aBook().isbn("9789999999991").build());

    assertThat(books.findByIsbn("9789999999991")).isPresent();
}

@Test
void listsTheBooksRunningLowWithTheLowestStockFirst() {
    entities.persist(aBook().isbn("9789999999992").stock(1).build());
    entities.persist(aBook().isbn("9789999999993").soldOut().build());
    entities.persistAndFlush(aBook().isbn("9789999999994").stock(50).build());

    assertThat(books.runningLow(2)).extracting(Book::getIsbn)
        .containsSequence("9789999999993", "9789999999992") // and the seeded
        ↪ sold-out Java Puzzlers
        .doesNotContain("9789999999994");
}
}

```

- @DataJpaTest runs every test in a transaction that is **rolled back** at the end. Tests do not see each other's data.
- TestEntityManager writes the test data directly, so the test checks only the query under test.
- The database is a real PostgreSQL (Testcontainers), not H2: a JPQL query with order by, a CHECK constraint or a PostgreSQL-specific type behave exactly as in production.

3.5 JSON Slice: @JsonTest

Money should not become a floating-point number in JSON. The record writes it as a string:

```

public record OrderConfirmation(String isbn, String title, int quantity,
                               @JsonFormat(shape = JsonFormat.Shape.STRING) BigDecimal
                               ↪ total,
                               String paymentId) {
}

```

```
@JsonTest
class OrderConfirmationJsonTest {

    @Autowired
    JacksonTester<OrderConfirmation> json;

    @Test
    void writesMoneyAsString() throws Exception {
        var confirmation = new OrderConfirmation("9780134685991", "Effective Java", 2,
            new BigDecimal("179.80"), "pay-7");

        assertThat(json.write(confirmation))
            .extractingJsonPathStringValue("$.total").isEqualTo("179.80");    // not
            ↪ 179.8
    }

    @Test
    void readsTheSameFormatBack() throws Exception {
        var read = json.parseObject("""
            {"isbn":"9780134685991","title":"Effective
            ↪ Java","quantity":2,"total":"179.80","paymentId":"pay-7"}""");

        assertThat(read.total()).isEqualByComparingTo("179.80");
    }
}
```

@JsonTest uses the same Jackson configuration as the application, so a changed JsonMapper setting would show up here.

3.6 Integration Tests: @SpringBootTest

```
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
@AutoConfigureRestTestClient
@Import({TestcontainersConfiguration.class})
class OrderFlowIT {

    @Autowired
    RestTestClient http;

    @Autowired
    BookRepository books;

    @MockitoBean                                // the external system: never called
    ↪ for real in tests
    PaymentClient payments;
```

```

@MockitoSpyBean                                     // the real bean, but its calls can be
↳ verified
AuditLog audit;

@Test
void anOrderIsPaidStoredAndAudited() {
    given(payments.charge(any())).willReturn(new PaymentReceipt("pay-42", "PAID"));
    int stockBefore = stockOf("9780134685991");

    http.post().uri("/api/orders").contentType(MediaType.APPLICATION_JSON)
        .body("""
                {"isbn": "9780134685991", "quantity": 2}""")
        .exchange()
        .expectStatus().isCreated()
        .expectBody().jsonPath("$.paymentId").isEqualTo("pay-42");

    assertThat(stockOf("9780134685991")).isEqualTo(stockBefore - 2);
    then(audit).should().record("ordered 2 × 9780134685991 for 179.80");
}

@Test
void aFailedPaymentRollsTheStockBack() {
    given(payments.charge(any())).willThrow(new RestClientException("payment service
    ↳ down"));
    int stockBefore = stockOf("9781617297571");

    http.post().uri("/api/orders").contentType(MediaType.APPLICATION_JSON)
        .body("""
                {"isbn": "9781617297571", "quantity": 1}""")
        .exchange()
        .expectStatus().is5xxServerError();

    assertThat(stockOf("9781617297571")).isEqualTo(stockBefore); // @Transactional
    ↳ rolled back
}

```

- RANDOM_PORT starts the real server. RestTestClient (Spring Framework 7) sends real HTTP requests.
- @MockitoBean PaymentClient replaces the one external system. Everything else is real, including PostgreSQL and the transaction.
- @MockitoSpyBean AuditLog keeps the real bean but lets the test verify its calls.
- The second test proves something no unit test can: when the payment fails, @Transactional rolls back the stock change.

★ Tip

Every distinct combination of `@MockitoBeans` and properties creates a new application context. Keep them the same across test classes, so that Spring can reuse one cached context.

3.7 Testcontainers

```
@TestConfiguration(proxyBeanMethods = false)
public class TestcontainersConfiguration {

    static final PostgreSQLContainer POSTGRES = new PostgreSQLContainer(
        DockerImageName.parse("pgvector/pgvector:0.8.6-
↪ pg18").asCompatibleSubstituteFor("postgres"))
        .withReuse(true);           // only if ~/.testcontainers.properties says
        ↪ testcontainers.reuse.enable=true

    @Bean
    @ServiceConnection              // Boot derives the DataSource (and Flyway) settings
    ↪ from the container
    PostgreSQLContainer postgresContainer() {
        return POSTGRES;
    }
}
```

- `static final`: one container per JVM, shared by all cached test contexts.
- `@ServiceConnection`: Boot reads the URL, user and password from the container. No `@DynamicPropertySource` is needed.
- `withReuse(true)` keeps the container running **between** test runs, but only if you enable it on your machine in `~/.testcontainers.properties` with `testcontainers.reuse.enable=true`. On CI it has no effect. That makes local test runs start in seconds.

3.8 Architecture Tests with ArchUnit

```
@AnalyzeClasses(packages = "com.springbootedu.testing", importOptions =
↪ ImportOption.DoNotIncludeTests.class)
class ArchitectureTest {

    @ArchTest
    static final ArchRule controllersDoNotTalkToRepositories = noClasses()
        .that().areAnnotatedWith(RestController.class)
```

```
        .should().dependOnClassesThat().haveSimpleNameEndingWith("Repository")
        .because("controllers go through a service, which owns the transaction");

@ArchTest
static final ArchRule pricingIsPlainJava = noClasses()
    .that().resideInAPackage("..pricing..")
    .should().dependOnClassesThat().resideInAPackage("org.springframework..")
    .because("business rules must be testable without Spring");

@ArchTest
static final ArchRule noFieldInjection = fields()
    .should().notBeAnnotatedWith(Autowired.class)
    .because("constructor injection makes dependencies explicit and classes
        ↪ testable with new");
}
```

ArchUnit reads the compiled classes and checks the rules like any other test. When a controller starts to use a repository directly, the build fails with a message such as:

```
Architecture Violation [Priority: MEDIUM] - Rule 'no classes that are annotated with
↪ @RestController should depend on
classes that have simple name ending with 'Repository', because controllers go through a
↪ service, which owns the transaction'
```

3.9 Test Data with Builders

```
public final class TestBooks {

    private String isbn = "9780000000000";
    private String title = "A Test Book";
    private BigDecimal price = new BigDecimal("10.00");
    private int stock = 5;

    private TestBooks() {
    }

    public static TestBooks aBook() {
        return new TestBooks();
    }

    public TestBooks isbn(String isbn) {
        this.isbn = isbn;
        return this;
    }
}
```

```
public TestBooks price(String price) {
    this.price = new BigDecimal(price);
    return this;
}

public TestBooks stock(int stock) {
    this.stock = stock;
    return this;
}

public TestBooks soldOut() {
    return stock(0);
}

public Book build() {
    return new Book(isbn, title, price, stock);
}
```

`aBook().stock(0).build()` says exactly what matters for the test. Everything else has a sensible default. When `Book` gets a new field, only the builder changes, not every test.

4. Common Mistakes and Best Practices

!! Caution

Never let a test depend on the current date, the order of a `HashSet`, a fixed `Thread.sleep` or the order in which tests run. Such tests are **flaky**: they fail from time to time without a code change, and soon nobody trusts the suite. Inject a `Clock`, assert without order, wait with `Awaitility`, and make every test prepare its own data.

- **Do:** test behaviour through public methods, not private details.
- **Don't:** use `@SpringBootTest` for everything. It is the slowest level.
- **Do:** use a real database (Testcontainers) for repository tests. H2 behaves differently from PostgreSQL.
- **Don't:** mock what you own and can run cheaply. A mocked `PriceCalculator` would test nothing.
- **Do:** give tests names that read as a requirement: `aFailedPaymentRollsTheStockBack`.
- **Don't:** assert more than the test is about. Overspecified tests break on every harmless change.

5. Summary

- The test pyramid: many fast unit tests, some slice tests, a few integration tests.
- Unit tests need no Spring. Mockito replaces collaborators, and ArgumentCaptor checks what was passed.
- `@WebMvcTest`, `@DataJpaTest` and `@JsonTest` each load one layer with its real configuration.
- `@SpringBootTest` with `RestTestClient` tests the whole application. `@MockitoBean` replaces external systems, `@MockitoSpyBean` observes real beans.
- Testcontainers with `@ServiceConnection` gives every test a real database, and `withReuse` makes it fast locally.
- Builders keep test data short, and ArchUnit turns architecture decisions into tests.

Next step: [Exercises](#)

6. Further Reading

- [Spring Boot — Testing · Testing Spring Boot Applications](#)
- [Spring Framework — MockMvcTester · RestTestClient · Bean Overriding](#)
- [JUnit User Guide · Mockito · AssertJ](#)
- [Testcontainers for Java · Reusable Containers](#)
- [ArchUnit User Guide](#)
- [Martin Fowler — The Practical Test Pyramid](#)